



Alfa Curriculum Maker

*Guia completo de estudo: cada linha explicada,
como implementar e o porquê de cada decisão.*

React 19 · TypeScript · Vite 8 · CSS puro

Documento didático para desenvolvedores Júnior

Sumário

1. Visão geral do projeto e da stack
2. Preparando o ambiente (passo a passo)
3. Anatomia das pastas do projeto
4. `src/types.ts` — os contratos de dados (linha por linha)
5. `src/data/steps.ts` — o roteiro da conversa
6. `Header.tsx` — cabeçalho e barra de progresso
7. `ChatMessage.tsx` — bolha de mensagem
8. `TypingIndicator.tsx` — animação "digitando"
9. `SuggestionChips.tsx` — botões de sugestão
10. `ChatInput.tsx` — campo de digitação
11. `SummaryCard.tsx` — resumo final + download
12. `App.tsx` — o cérebro: máquina de estados do chat
13. CSS: tema vermelho sangue explicado
14. Rodar, buildar e validar o projeto
15. Próximos passos: backend, PDF real e testes
16. Glossário essencial para devs juniores

COMO USAR ESTE GUIA

Leia cada capítulo com o arquivo correspondente aberto no seu editor. Sempre que vir um bloco de código, tente responder antes de ler a explicação: "o que essa linha faz?". Depois compare. Os quadros **Como implementar**, **Por que assim** e **Erro comum** aparecem nos pontos onde juniores mais travam.

1. Visão geral do projeto

O **Alfa Curriculum Maker** é um web app de conversa (estilo chat) onde o usuário responde perguntas e, ao final, recebe um currículo montado. A identidade visual segue a pegada do AlfaPDFReader: **fundo escuro com tons de vermelho sangue**.

A stack escolhida e o porquê de cada peça

Tecnologia	Papel	Por quê?
Vite 8	Bundler + servidor de desenvolvimento	Sobe em milissegundos, recarrega a página ao salvar (HMR) e já vem configurado com TypeScript. Alternativas antigas (Create React App) estão obsoletas.
React 19	Biblioteca de interface	Ideal para apps orientados a estado — exatamente o caso de um chat: cada resposta muda a tela.
TypeScript 6	Type safety	Erros de digitação e de formato são pegos ANTES de rodar. Para quem está começando, o TS funciona como um professor que corrige na hora.
CSS puro	Estilização	Sem Tailwind/styled-components de propósito: você aprende fundamentos que servem em qualquer projeto.
oxlint	Linter	Pega código suspeito (variável não usada, bugs clássicos) sem configuração.

O fluxo do app em uma frase

O app mantém uma lista de mensagens (*chat*) e uma lista de etapas (*roteiro*). Quando o usuário responde uma etapa, salvamos a resposta no objeto `resume`, avançamos o índice da etapa e o assistente faz a próxima pergunta. Na última etapa mostramos o resumo e o botão de download.

CONCEITO-CHAVE: ESTADO DIRIGE A INTERFACE

Nada aqui manipula o DOM diretamente (nada de `document.getElementById(...)` para trocar texto). Mudamos objetos de estado (`useState`) e o React redesenha a tela. Esse é o modelo mental do React: **UI = f(estado)**. Grave isso.

2. Preparando o ambiente

Pré-requisitos

- **Node.js 18+** (usamos a v24). Verifique no terminal: `node --version`
- **npm** (vem junto com o Node): `npm --version`
- VS Code (recomendado) + extensão oficial ESLint/oxlint opcional

Criação do projeto (o que realmente aconteceu)

```
# cria o projeto na pasta atual usando o template react + typescript
npm create vite@latest . -- --template react-ts

# instala as dependências listadas no package.json
npm install
```

Comando	Para que serve
<code>npm run dev</code>	Sobe servidor local em <code>http://localhost:5173</code> com recarga automática. Use durante o desenvolvimento.
<code>npm run build</code>	Roda o TypeScript (<code>tsc -b</code>) e gera a versão otimizada na pasta <code>dist/</code> . Se tiver erro de tipo, o build falha — é proposital.
<code>npm run lint</code>	Analisa o código procurando problemas (variáveis mortas, erros lógicos).
<code>npm run preview</code>	Serve a pasta <code>dist/</code> localmente para você testar o build de produção antes de publicar.

POR QUE O BUILD RODA O TYPESCRIPT ANTES DO VITE?

Olhe o `package.json` : `"build": "tsc -b && vite build"` . O `&&` significa "só continue se o anterior teve sucesso". Ou seja: nenhum código com erro de tipo chega a ser empacotado. É sua primeira barreira de qualidade automatizada.

3. Anatomia das pastas

```
AlfaCurriculumMaker/
├─ index.html           # página HTML única (SPA) — carrega src/main.tsx
├─ package.json         # dependências + scripts npm
├─ tsconfig*.json       # regras do TypeScript (app e node separados)
├─ vite.config.ts       # config do Vite (plugin React)
├─ public/
│  └─ favicon.svg       # ícone da aba do navegador
└─ src/
   ├─ main.tsx          # porta de entrada: monta o App no DOM
   ├─ index.css         # reset global + variáveis do tema vermelho
   ├─ App.tsx           # cérebro: estado do chat + composição dos componentes
   ├─ App.css           # estilos dos componentes do chat
   ├─ types.ts          # contratos de dados (interfaces/types)
   ├─ data/
   │  └─ steps.ts       # roteiro das perguntas + sugestões
   └─ components/
      ├─ Header.tsx     # logo + barra de progresso
      ├─ ChatMessage.tsx # bolha do bot ou do usuário
      ├─ TypingIndicator.tsx # três pontinhos animados
      ├─ SuggestionChips.tsx # chips clicáveis com exemplos
      ├─ ChatInput.tsx  # input + botão enviar
      └─ SummaryCard.tsx # resumo final + download .txt
```

Regras de organização (memorize estas)

- **Componentes recebem dados via props e disparam eventos para cima.** Um componente nunca altera o estado do pai diretamente — ele chama uma função passada pelo pai.
- **Dados de conteúdo ficam longe da UI.** As perguntas vivem em `data/steps.ts`. Se amanhã você quiser editar o texto de uma pergunta, não vai caçar dentro de JSX.
- **Tipos compartilhados ficam em `types.ts`.** Importados por todos, evita que cada arquivo invente um formato diferente.

COMO IMPLEMENTAR (PADRÃO DE COMPONENTE)

Todo componente deste projeto segue a mesma receita: 1) defina a **interface das props** acima do componente; 2) exporte uma função com o mesmo nome do arquivo; 3) retorne JSX; 4) estilos ficam no CSS central com classes prefixadas pelo nome do bloco (ex.: `.header__logo` — padrão BEM simplificado).

4. src/types.ts — os contratos de dados

Este arquivo é pequeno mas é a fundação de tudo. Comece sempre pelos tipos quando estiver planejando uma feature: eles são a documentação viva do domínio.

Linha por linha

```
export type Sender = 'bot' | 'user';
// Union type: Sender só pode ser 'bot' OU 'user'.
// Por quê? Evita booleanos ambíguos tipo isBot/isUser duplicados.
// Erro comum: usar string livre – aí alguém digita 'Bot' com B maiúsculo e quebra o CSS.

export interface Message {
  id: number;      // identificador único – usamos como key na lista do React
  from: Sender;    // quem enviou (define a cor da bolha)
  text: string;    // conteúdo
}

export interface ResumeData {
  fullName: string; // nome completo
  targetRole: string; // vaga desejada
  layout: string;    // estilo visual escolhido (clássico/moderno/minimalista)
  contact: string;   // email/telefone/cidade/linkedin
  summary: string;   // resumo profissional
  experience: string; // experiências
  education: string; // formação
  skills: string;    // habilidades
  languages: string; // idiomas (opcional)
}
// Por quê strings simples e não arrays/objetos complexos?
// Decisão de escopo: no MVP o usuário digita texto corrido.
// Quando integrar backend, experience pode virar Experience[] – e só este arquivo muda.

export type ResumeField = keyof ResumeData;
// keyof cria um union com as chaves do ResumeData:
// 'fullName' | 'targetRole' | ... | 'languages'
// Uso: permitir acesso dinâmico seguro – obj[campo] sem perder type safety.

export interface ChatStep {
  id: ResumeField; // qual campo do currículo esta etapa preenche
  question: string; // pergunta exibida no chat
  placeholder: string; // texto de ajuda dentro do input
  suggestions: string[]; // chips de exemplo
  optional?: boolean; // ? = propriedade OPCIONAL (pode ser undefined)
}

export const EMPTY_RESUME: ResumeData = { ... };
```

```
// Objeto inicial com todas as strings vazias.  
// Por quê? Evita undefined espalhado: todo campo existe desde o início.  
// Usamos no useState inicial e no botão "Recomeçar".  
  
export const SKIP_VALUE = '__skip__';  
// Valor-sentinela: identifica internamente que o usuário pulou a etapa.  
// Nunca aparece na tela; é traduzido para '' antes de salvar.
```

ERRO COMUM DE JÚNIOR

Declarar tipos duplicados em vários arquivos ("interface IMessage aqui, outra ali"). Regra: **um conceito de domínio = um tipo**, definido uma vez e importado. Se dois lugares precisam, ambos importam de `types.ts`.

5. src/data/steps.ts — o roteiro da conversa

Aqui vive o "script" do assistente. O app é genérico: ele percorre o array `STEPS` do início ao fim, sem saber quais são as perguntas. Isso significa que **you** pode adicionar/remeter/reordenar perguntas sem tocar em nenhuma linha de lógica.

```
import type { ChatStep } from '../types';
// import type = importa APENAS o tipo, some do bundle final.
// Boa prática obrigatória quando tsconfig usa verbatimModuleSyntax.

export const WELCOME_MESSAGE =
  'Bem-vindo ao Alfa Curriculum Maker! ...';

export const STEPS: ChatStep[] = [
  {
    id: 'fullName',
    question: 'Qual é o seu nome completo?',
    placeholder: 'Digite seu nome completo...',
    suggestions: ['Meu nome é Maria Oliveira Santos', 'João Pedro Almeida'],
  },
  ...
  {
    id: 'languages',
    question: 'Por último: você fala algum idioma além do português?',
    placeholder: 'Ex.: Inglês intermediário',
    optional: true,
    suggestions: ['Inglês intermediário', 'Pular esta etapa'],
  },
];
```

Decisão	Motivo técnico
Array ordenado (não objeto)	A ordem das perguntas importa. Percorrer array por índice (<code>STEPS[stepIndex]</code>) é trivial.
<code>suggestions: []</code> permitido	Nem toda pergunta precisa de chips. O componente de chips simplesmente não renderiza nada.
<code>optional?: true</code> apenas em idiomas	Habilita o chip "Pular esta etapa", renderizado condicionalmente.
Textos completos fora do JSX	Permite futura i18n (tradução) e edição por não-programadores.

EXERCÍCIO SUGERIDO

Adicione uma nova etapa entre formação e habilidades chamada `certifications` . Você precisará: (1) criar o campo em `ResumeData` ; (2) adicionar o item em `STEPS` ; (3) adicionar o rótulo em `FIELD_LABELS` dentro do `SummaryCard` . Se fez só isso, o sistema inteiro já funciona — esse é o sinal de bom design.

6. Header.tsx — cabeçalho e progresso

```
interface HeaderProps {
  step: number;          // etapa ATUAL (começa em 1)
  totalSteps: number;    // total de etapas (STEPS.length)
}

export function Header({ step, totalSteps }: HeaderProps) {
  const progress = Math.round((step / totalSteps) * 100);

  return (
    <header className="header">
      <div className="header__brand">
        <span className="header__logo">A</span>
        ...
      </div>
      <div className="header__progress" aria-label={`Etapa ${step} de ${totalSteps}`}>
        <div className="header__progress-bar">
          <div className="header__progress-fill" style={{ width: `${progress}%` }} />
        </div>
      </div>
    </header>
  );
}
```

- `interface HeaderProps` — contrata as props. Se o pai esquecer uma prop obrigatória, o TS acusa no ato.
- `Math.round((step / totalSteps) * 100)` — regra de três para percentual. Ex.: 3 de 9 etapas → 33%.
- `aria-label` com template string — acessibilidade: leitores de tela anunciam a etapa. Custa 1 linha e melhora o produto.
- `style={{ width: ... }}` — chaves duplas: a externa marca "expressão JS", a interna é o objeto literal. A largura anima graças ao `transition: width .4s` no CSS.

7. ChatMessage.tsx — bolha de mensagem

```
export function ChatMessage({ message }: ChatMessageProps) {
  const isBot = message.from === 'bot';

  return (
    <div className={`chat-message ${isBot ? 'chat-message--bot' : 'chat-message--user'}`}>
      {isBot && <span className="chat-message__avatar">A</span>}
      <div className="chat-message__bubble">
        {message.text.split('\n').map((line, index) => (
```

```

        <p key={index}>{line || '\u00A0'}</p>
      )}}
    </div>
  </div>
);
}

```

- **Renderização condicional:** `{isBot && ...}` — se for mensagem do usuário, nem renderiza avatar (o CSS alinha à direita com `flex-direction: row-reverse`).
- **Split por \n:** as perguntas têm quebras de linha. JSX ignora `\n`, então transformamos cada linha em parágrafo. Padrão importante para qualquer texto multilinha em React.
- `{line || '\u00A0'}` — linha vazia vira espaço não-quebrável (nbsp), senão o parágrafo colapsa e o espaçamento desaparece. Detalhe fino que separa júnior de pleno.
- **key={index}:** aceitável aqui porque a lista nunca reordena. Em listas mutáveis use IDs estáveis.

ERRO COMUM DE JÚNIOR

Colocar `dangerouslySetInnerHTML` para resolver quebras de linha. Isso abre brecha XSS quando o texto vem do usuário. Transformar em elementos seguros (como fizemos) é o caminho certo.

8. TypingIndicator.tsx — "digitando..."

```
export function TypingIndicator() {
  return (
    <div className="chat-message chat-message--bot">
      <span className="chat-message__avatar">A</span>
      <div className="chat-message__bubble typing" aria-label="Assistente digitando">
        <span className="typing__dot" />
        <span className="typing__dot" />
        <span className="typing__dot" />
      </div>
    </div>
  );
}
```

- Reaproveita as classes de `chat-message--bot` para ficar visualmente idêntico a uma mensagem — só o conteúdo muda (pontinhos).
- A animação é 100% CSS (`@keyframes typing-bounce` com delays escalonados via `:nth-child`). Zero JS: mais performático.
- Quem decide mostrar/esconder é o App, via estado `isTyping` . O componente é burro de propósito (apresentação pura).

9. SuggestionChips.tsx — sugestões clicáveis

```
export function SuggestionChips({ suggestions, optional, onPick }: SuggestionChipsProps) {
  if (suggestions.length === 0 && !optional) return null;
  // Early return: componente decide sozinho quando não deve existir.
  // return null é a forma idiomática de "renderize nada".

  return (
    <div className="suggestion-chips">
      {suggestions.map((text) => (
        <button key={text} type="button" onClick={() => onPick(text)}>{text}</button>
      ))}
      {optional && (
        <button className="chip--skip" onClick={() => onPick(SKIP_VALUE)}>
          Pular esta etapa
        </button>
      )}
    </div>
  );
}
```

- `type="button"` — SEMPRE em botões dentro de forms no React. O default é `submit`, e um clique num chip dispararia o submit do formulário vizinho... na verdade chips estão fora do form, mas o hábito evita surpresas quando alguém mover o JSX.
- `onPick(SKIP_VALUE)` — o chip de pular envia o valor-sentinela. Quem sabe o que fazer com ele é o App (único dono da lógica). Isso mantém o componente reutilizável.
- **Inversão de controle:** o componente NÃO avança etapas, NÃO salva nada. Ele apenas grita "o usuário escolheu X". O pai decide. Esse desenho torna testes triviais.

10. ChatInput.tsx — campo de digitação

```
<form onSubmit={(event) => {
  event.preventDefault(); // impede reload da página (comportamento default do form)
  onSend();
}}>
  <input
    value={value} // componente CONTROLADO: o valor vem do pai
    onChange={(e) => onChange(e.target.value)} // ...e cada tecla sobe pro pai
    disabled={disabled}
    placeholder={placeholder}
    aria-label="Sua resposta"
  />
  <button type="submit" disabled={disabled || value.trim() === ''}>...</button>
</form>
```

O conceito mais importante deste arquivo: Controlled Component

No React, o input tem DUAS fontes de verdade possíveis: o DOM interno dele, ou o seu estado. Escolhemos o estado: o valor exibido é `value={value}` (que veio de `draft` no App) e cada tecla chama `onChange`, que atualiza o estado, que redesenha o input. Ciclo fechado.

- **Por que o estado mora no App e não no input?** Porque o botão "enviar" do App precisa LER o texto. Estado sempre no ancestral comum de quem lê e quem mostra. Isso é "lifting state up".
- `disabled || value.trim() === ''` — botão desliga quando vazio ou quando o bot está digitando. `.trim()` impede enviar só espaços.
- O botão de enviar é um SVG inline (seta) em vez de emoji: consistência entre sistemas operacionais.
- Enter no input submete automaticamente porque estamos dentro de um `<form>` com botão submit — de graça, sem JS extra.

ERRO COMUM DE JÚNIOR

Guardar o texto num `useRef` ou ler `e.target.value` direto no send. Funciona até o dia em que você precisa limpar o campo programaticamente ou desabilitar o botão conforme o conteúdo. Com controlled component, isso é uma linha.

11. SummaryCard.tsx — resumo + download

```
function buildResumeText(resume: ResumeData): string {
  const lines = FIELD_LABELS.map(({ key, label }) =>
```

```

    `${label}: ${resume[key] || '(nao informado)'} `);
    return ['CURRICULO - ALFA CURRICULUM MAKER', '====', '', ...lines].join('\n');
}

function downloadResume(resume: ResumeData) {
    const blob = new Blob([buildResumeText(resume)], { type: 'text/plain;charset=utf-8' });
    const url = URL.createObjectURL(blob); // gera endereço temporário na memória
    const anchor = document.createElement('a'); // âncora fantasma
    anchor.href = url;
    anchor.download = 'curriculo-alfa.txt'; // atributo download força baixar
    anchor.click();
    URL.revokeObjectURL(url); // libera memória - SEMPRE faça isso
}

```

- `FIELD_LABELS` — array único ligando chave técnica → rótulo humano. Serve tanto para a tela quanto para o arquivo. Uma fonte de verdade.
- Técnica do download: Blob (arquivo em memória) + object URL + âncora invisível. É o padrão universal de download client-side, sem backend.
- Funções auxiliares FORA do componente: não dependem de estado, então não precisam ser recriadas a cada render.
- Campos vazios exibem "Não informado" — o `||` trata string vazia como falsy.

12. App.tsx — o cérebro do chat

Este é o arquivo que amarra tudo. Leia com calma: cada linha está comentada.

a) Estado e refs

```
const [messages, setMessages] = useState<Message[]>(() => [
  { id: 0, from: 'bot', text: WELCOME_MESSAGE },
]);
// Initializer LAZY: a função roda UMA vez, na criação.
// Por quê assim e não useEffect? O React 18+/StrictMode executa efeitos 2x em dev,
// o que duplicaria a mensagem de boas-vindas. Estado inicial não sofre desse mal.

const [stepIndex, setStepIndex] = useState(0); // qual pergunta está ativa
const [resume, setResume] = useState<ResumeData>(EMPTY_RESUME); // respostas
const [isTyping, setIsTyping] = useState(false); // mostra indicador "digitando"
const [draft, setDraft] = useState(''); // texto sendo digitado agora

const nextIdRef = useRef(1); // contador de IDs (não causa re-render)
const timerRef = useRef<number | null>(null); // guarda o setTimeout ativo
const bottomRef = useRef<HTMLDivElement | null>(null); // aponta pro fim da lista
```

QUANDO USAR REF EM VEZ DE STATE?

Estado (`useState`) = dado que aparece na tela; mudou → re-renderiza. Ref (`useRef`) = dado que o render não enxerga (IDs, timers, elementos DOM). Se colocasse o contador de ID em `useState`, teria renders desnecessários e risco de loop.

b) Funções centrais

```
function pushMessage(from, text) {
  setMessages((current) => [...current, { id: nextIdRef.current, from, text }]);
  // Spread + novo item = IMUTABILIDADE. Nunca faça messages.push()!
  // O React compara referências: array novo → detectou mudança → re-render.
  nextIdRef.current += 1;
}

function botSay(text: string) {
  setIsTyping(true); // pontinhos aparecem
  timerRef.current = window.setTimeout(() => {
    setIsTyping(false); // pontinhos saem
    pushMessage('bot', text); // resposta chega
  }, 1000);
}
```



```
    }, BOT_DELAY_MS); // 900ms: parece que ele "digita"
  }
}
```

c) Envio de mensagem (a transição de estados)

```
function handleSend(rawText: string) {
  const isSkip = rawText === SKIP_VALUE;
  const text = isSkip ? 'Pular esta etapa' : rawText.trim();
  if (text === '' || !currentStep || finished || isTyping) return;
  // GUARDA DE SEGURANÇA: bloqueia cliques duplos e envios fora de hora.

  pushMessage('user', text);
  setResume((prev) => ({ ...prev, [currentStep.id]: isSkip ? '' : rawText.trim() }));
  // [currentStep.id] = computed property: grava no campo CERTO do resume
  // (fullName na etapa 1, contact na etapa 4...) sem if/switch gigante.

  const nextQuestion = stepIndex + 1 < STEPS.length
    ? STEPS[stepIndex + 1].question
    : FINISH_MESSAGE;
  setStepIndex((index) => index + 1);
  setDraft(''); // limpa o input
  botSay(nextQuestion); // assistente pergunta a próxima
}
```

d) Efeitos: início e auto-scroll

```
useEffect(() => {
  if (didStartRef.current) return; // guard contra StrictMode rodar 2x em dev
  didStartRef.current = true;
  timerRef.current = setTimeout(() => pushMessage('bot', STEPS[0].question), BOT_DELAY_MS);
  return () => clearTimeout(timerRef.current); // cleanup: cancela timer se o App sumir
}, []); // [] = roda só após a primeira montagem

useEffect(() => {
  bottomRef.current?.scrollIntoView({ behavior: 'smooth' });
  // ?. = optional chaining: se a ref ainda não existe, não explode.
}, [messages, isTyping]); // roda sempre que mensagens/typing mudam
```

O FLUXO COMPLETO EM DIAGRAMA

```
usuário digita → ChatInput.onChange → setDraft (estado sobe pro App)
Enter/clique → handleSend(draft)
                  |
                  |─ pushMessage('user')      ► mensagem entra no chat
                  |─ setResume({campo: resp}) ► resposta arquivada
                  |─ setStepIndex(i+1)       ► avança etapa
```

```
└─ botSay(proxima)           ► isTyping=true → 900ms → pergunta nova  
última etapa? →► stepIndex >= STEPS.length →► finished=true →► SummaryCard renderiza
```

e) Reinício limpo

```
function handleRestart() {  
  if (timerRef.current !== null) clearTimeout(timerRef.current); // mata timer pendente  
  setMessages([]); nextIdRef.current = 1; // zera histórico E o contador  
  setStepIndex(0); setResume(EMPTY_RESUME); setIsTyping(false); setDraft('');  
  pushMessage('bot', WELCOME_MESSAGE); // recomeça a conversa  
}
```

Repare: reiniciar é literalmente voltar os estados aos valores iniciais. Quando seu app é **função do estado**, reset é trivial. Essa é a recompensa do design correto.

13. CSS: o tema vermelho sangue

index.css — variáveis e reset

```
:root {
  --bg-900: #12040a;    // quase-preto arroxeadado: base do fundo
  --bg-800: #1d060d;
  --surface: #260810;   // superfícies (painéis, inputs)
  --surface-2: #320b16; // superfície hover/bolha do bot
  --border-red: #54121f; // bordas discretas
  --red-blood: #8a0f1e;  // vermelho sangue principal
  --red-bright: #c1121f; // vermelho vivo (gradientes, CTA)
  --red-glow: #ff4d5e;   // brilho/hover – dá o "neon sangue"
  --text-main: #f9eeec;  // branco levemente rosado (menos agressivo que #fff)
  --text-muted: #d5a3aa; // texto secundário rosado
}
body {
  background-image:
    radial-gradient(ellipse 90% 55% at 50% -10%, rgba(193,18,31,.35), transparent),
    radial-gradient(ellipse 70% 50% at 100% 110%, rgba(138,15,30,.28), transparent),
    linear-gradient(var(--bg-800), var(--bg-900));
  // 3 camadas empilhadas: dois halos de luz vermelha + gradiente vertical.
  // rgba(...) com transparência deixa o brilho suave em vez de mancha chapada.
}
```

- **Variáveis CSS (--nome)**: mudou a cor em UM lugar, mudou no app inteiro. Quer uma versão "Halloween"? Troque 3 variáveis.
- **Escalas 800/900**: convenção emprestada de design systems (tons de escuro para claro).

Padrões de layout usados

Trecho	O que faz	Por quê
<code>.app { height:100%; max-width:860px; margin:0 auto }</code>	App ocupa a altura toda, centrado, largura máxima	Chat precisa de coluna fixa; 860px é confortável para leitura
<code>.chat { flex:1; min-height:0 }</code>	Área de mensagens cresce e encolhe	<code>min-height:0</code> é O detalhe que faz overflow funcionar em flex children — esqueça isso e o scroll vaza da tela
<code>.chat-message--user { align-self:flex-end }</code>	Mensagens do usuário à direita	Padrão universal de apps de mensagem (WhatsApp-like)

bordas assimétricas (`border-bottom-left-radius:4px`)

"Rabinho" na bolha

Direção visual de quem fala

Animação dos pontinhos

```
.typing__dot { animation: typing-bounce 1s infinite ease-in-out; }  
.typing__dot:nth-child(2) { animation-delay: .15s; } // cada ponto atrasa um pouco  
.typing__dot:nth-child(3) { animation-delay: .30s; }  
  
@keyframes typing-bounce {  
  0%, 60%, 100% { transform: translateY(0); opacity:.4; } // repouso  
  30%           { transform: translateY(-5px); opacity:1; } // pico do pulo  
}
```

Delays escalonados criam a onda. Animação em `transform/opacity` é acelerada por GPU — nunca anime `top/left`.

ACESSIBILIDADE INCLUÍDA

`aria-label` no progresso, no input e no indicador; `:focus-within` ilumina o input (quem navega por teclado sabe onde está); contraste dos textos validado sobre o fundo escuro.

14. Rodar, buildar e validar

```
npm run dev      # http://localhost:5173 – desenvolva aqui
npm run lint     # oxlint analisa o código
npm run build    # tsc valida tipos + Vite gera dist/
npm run preview  # testa o build final localmente
```

Checklist de teste manual (faça sempre que mexer)

- Responder digitando texto livre em todas as etapas
- Clicar em cada chip de sugestão
- Usar "Pular esta etapa" em idiomas → o resumo mostra "Não informado"
- Tentar enviar mensagem vazia → botão desabilitado
- Clicar "Baixar currículo (.txt)" → arquivo sai com os dados
- Clicar "Recomeçar" → chat volta ao zero sem mensagens órfãs
- Redimensionar a janela para celular (DevTools F12) → layout empilha

DICA DE DEPURAÇÃO

Instale a extensão **React Developer Tools** no navegador. Ela mostra a árvore de componentes e os valores de `messages`, `stepIndex` e `resume` EM TEMPO REAL. Ver o estado mudando é a melhor forma de entender o fluxo.

15. Próximos passos (ordem sugerida)

1. Gerar PDF real (em vez de .txt)

```
npm install jspdf
// dentro do SummaryCard:
import { jsPDF } from 'jspdf';
const doc = new jsPDF();
doc.text(buildResumeText(resume), 10, 10);
doc.save('curriculo-alfa.pdf');
```

Por quê jsPDF? Roda 100% no navegador, sem backend. Quando o design do PDF precisar ser rico (cores, colunas), avalie `@react-pdf/renderer`, que monta o PDF como componentes React.

2. Conectar num backend real (substituir a "IA local")

```
// hoje: botSay(nextQuestion) responde localmente
// amanhã: envie as respostas e receba as perguntas de uma API:
async function askAssistant(resume: ResumeData) {
  const response = await fetch('/api/chat', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(resume),
  });
  if (!response.ok) throw new Error('Falha na API');
  return (await response.json()).message as string;
}
```

- Guarde a URL da API em variável de ambiente: crie `.env` com `VITE_API_URL=...` e leia com `import.meta.env.VITE_API_URL`. Variáveis precisam do prefixo `VITE_` para serem expostas.
- Trate loading e erro: mostre "Não consegui falar com o servidor, tente de novo" em vez de congelar o chat.

3. Salvar rascunho no localStorage

```
useEffect(() => {
  localStorage.setItem('alfa-resume-draft', JSON.stringify({ resume, stepIndex }));
}, [resume, stepIndex]);
// na inicialização, leia e restaure – usuário fecha a aba e não perde nada.
```

4. Testes automatizados

Instale Vitest + Testing Library. Primeiro teste sugerido: renderizar o App, digitar um nome, pressionar Enter e verificar que a segunda pergunta aparece. Depois: chip de pular, restart, download mockado. Testes de fluxo de estado são os que mais protegem este tipo de app.

5. Refino visual do currículo

Hoje o layout é uma preferência coletada (`layout`). Pleno: renderize 3 previews diferentes na SummaryCard aplicando classes condicionais (`summary-card--classic/modern/minimal`).

16. Glossário essencial

Termo	Tradução para júnior
Componente	Função que retorna tela (JSX) e reage às suas props. Como LEGO: peças combinam formam a página.
Props	Parâmetros do componente. Só descem (pai → filho). Filho comunica-se subindo callbacks (ex.: <code>onPick</code>).
<code>useState</code>	Memória do componente entre renders. Retorna [valor, setter]. Setter agenda novo render.
<code>useEffect</code>	"Depois de renderizar, execute isto". Array de deps controla QUANDO: <code>[]</code> = só na montagem; com deps = quando elas mudarem.
<code>useRef</code>	Caixinha mutável que sobrevive aos renders sem disparar nenhum. Para IDs, timers e elementos DOM.
Imutabilidade	Nunca altere array/objeto de estado no lugar. Crie cópia: spread <code>[...lista]</code> / <code>{...obj}</code> .
Union type	<code>'a' 'b'</code> : o valor só pode ser um desses. TS reclama se vier outra coisa.
<code>keyof</code>	Transforma as chaves de uma interface num union de nomes de campos.
Optional chaining (<code>?.</code>)	<code>a?.b</code> = se <code>a</code> existir, acesse <code>b</code> ; senão retorne undefined sem erro.
Computed property	<code>{ [chave]: valor }</code> : monta a chave do objeto dinamicamente. Base do <code>setResume</code> genérico.
Controlled component	Input cujo valor vem do estado e volta pra ele no <code>onChange</code> . Única fonte de verdade: o estado.
<code>StrictMode</code>	Modo de desenvolvimento que roda efeitos/render 2x para expor efeitos mal escritos. Não afeta produção.
SPA	Single Page Application: um HTML só, o JS troca o conteúdo. Nosso <code>index.html</code> nunca recarrega.
Blob	Dados binários em memória do navegador. Com <code>objectURL</code> + tag <code><a download></code> , vira arquivo baixável.

Early return

Sair da função no começo quando algo invalida (`if (!ok) return`). Reduz aninhamento e aumenta legibilidade.

PALAVRAS FINAIS DO PROFESSOR

Você acabou de ler a anatomia completa de um app React real. Os três pilares que sustentam este projeto são os mesmos de qualquer app grande: **(1) estado mínimo e bem localizado**, **(2) dados separados da apresentação**, **(3) componentes burros + um cérebro esperto**. Domine esses três e frameworks viram detalhe. Agora abra o terminal, rode `npm run dev`, quebre algo de propósito e conserte. É assim que se aprende de verdade.